

唐浚杰交接文档

唐浚杰 曾立强 | 今天修改

独立站

独立站是我们开发根据市场的需求联合ui制作的门户网站,担任展示品牌,售卖商品的功能

我们独立站搭载在shopify的平台之上,shopify一个低代码的编辑平台,所谓低代码就是我们编写代码以便市场能够根据我们提供的一些参数,让市场能够调整页面,比如我们把组件的宽高使用shopify提供的工具拆离出来,那么市场就能自己根据需求调整宽高等等。

开发前的准备

代码库

<https://gitee.com/tangiblefuture/looishopifyweb.git>

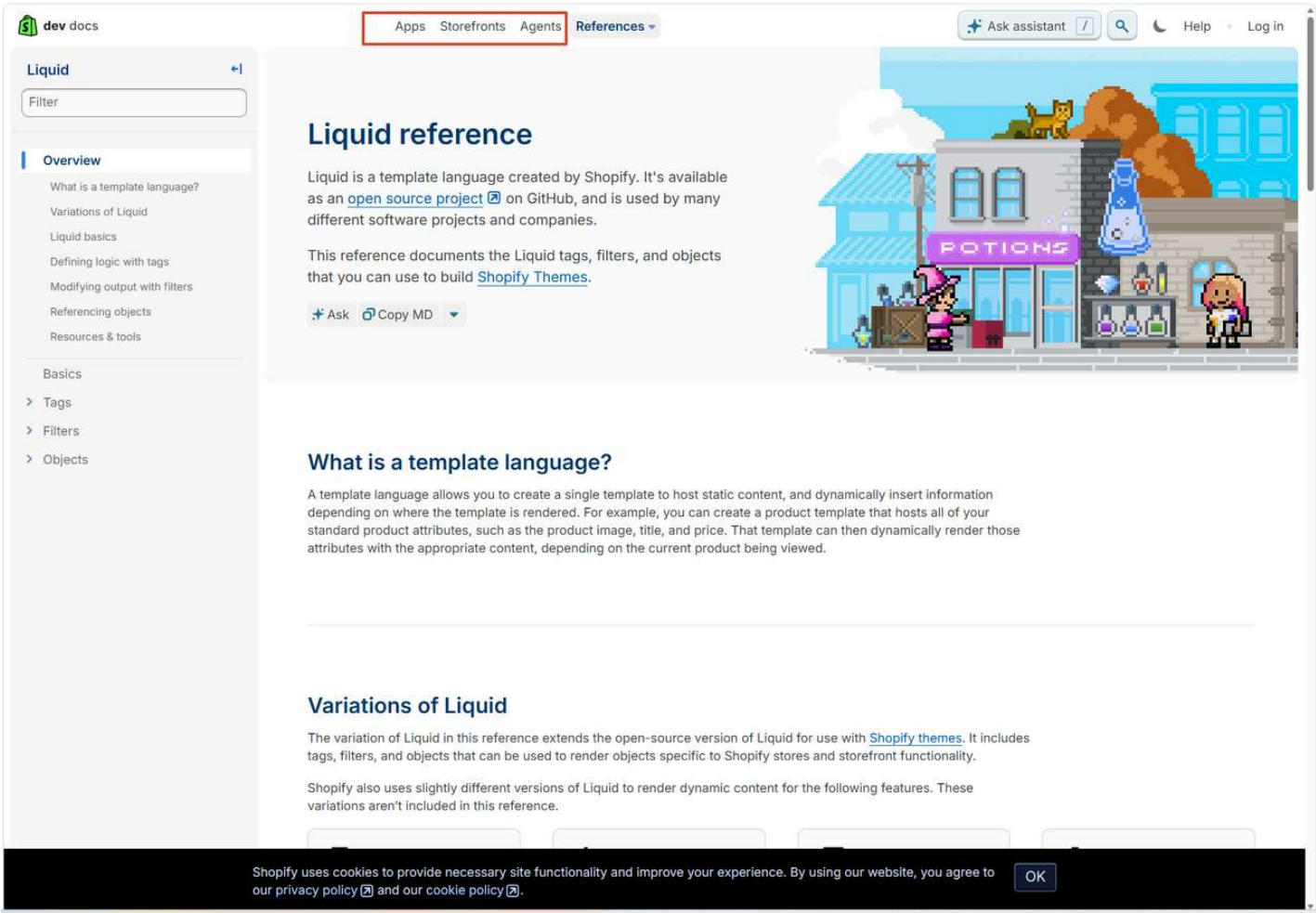
官方文档

总的文档, [Shopify Dev Docs](#)

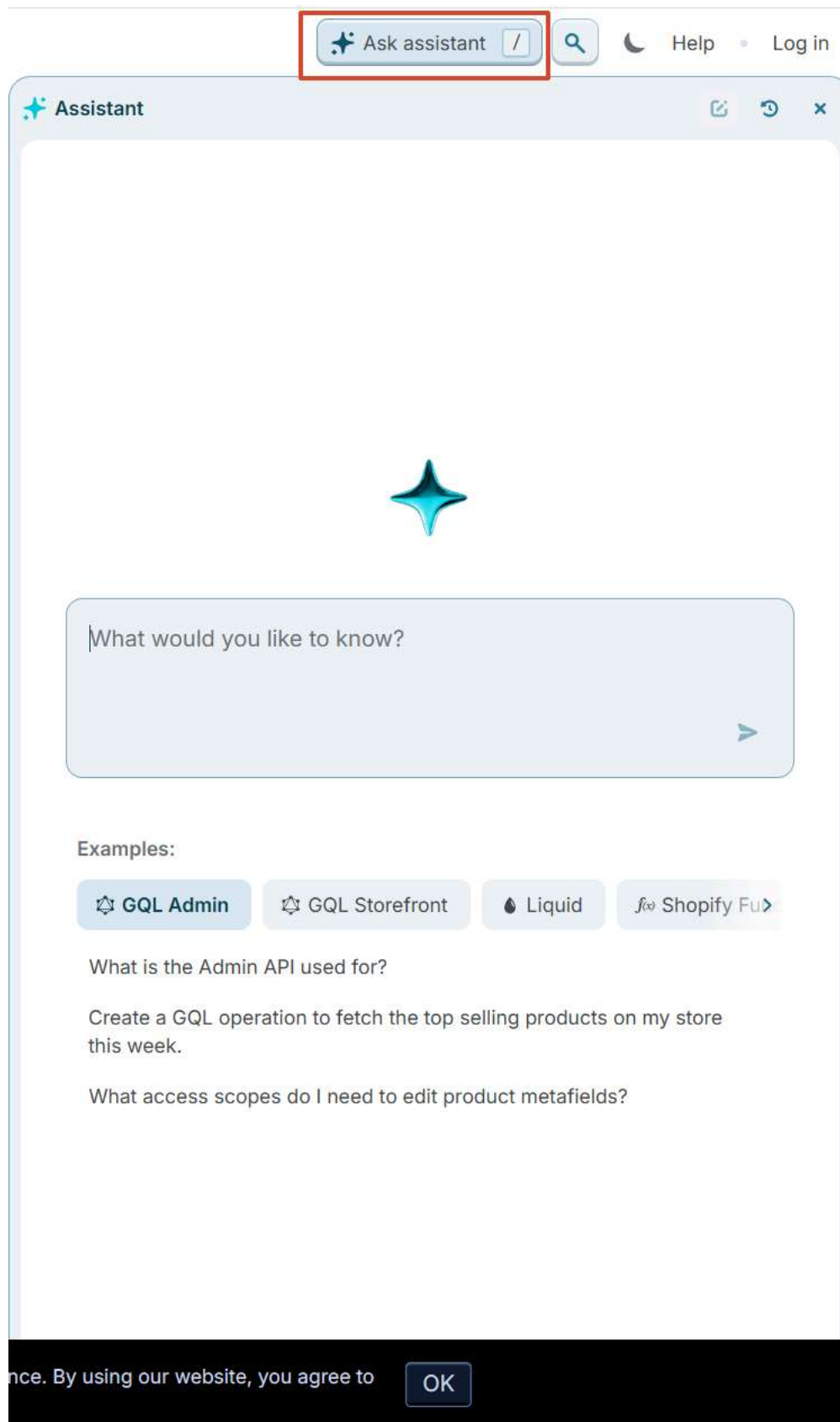
独立站主题开发文档[Themes](#)

liquid语法文档[Liquid reference](#)

每个文档都需要看, 这里需要花很长时间, 你可以自己创建一个自己的主题, 自己试验一下



左侧ask assistant可以向官方的ai询问问题,这是最高效的学习渠道



虽然有了官方文档，但是我还是介绍一下关于这个项目

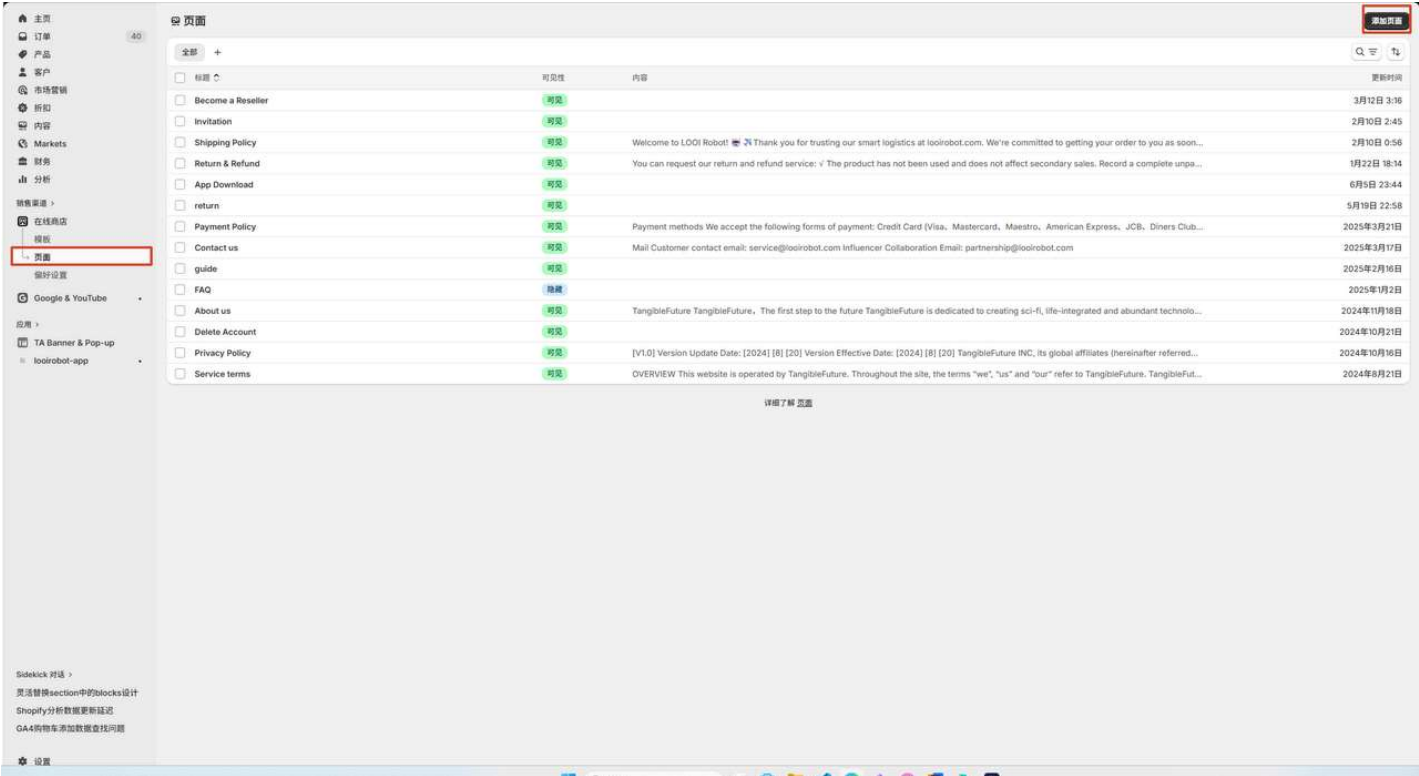
关于页面

首先关于页面的部分，页面分为两种，一种是官方自己的页面，另一种是自定义页面，

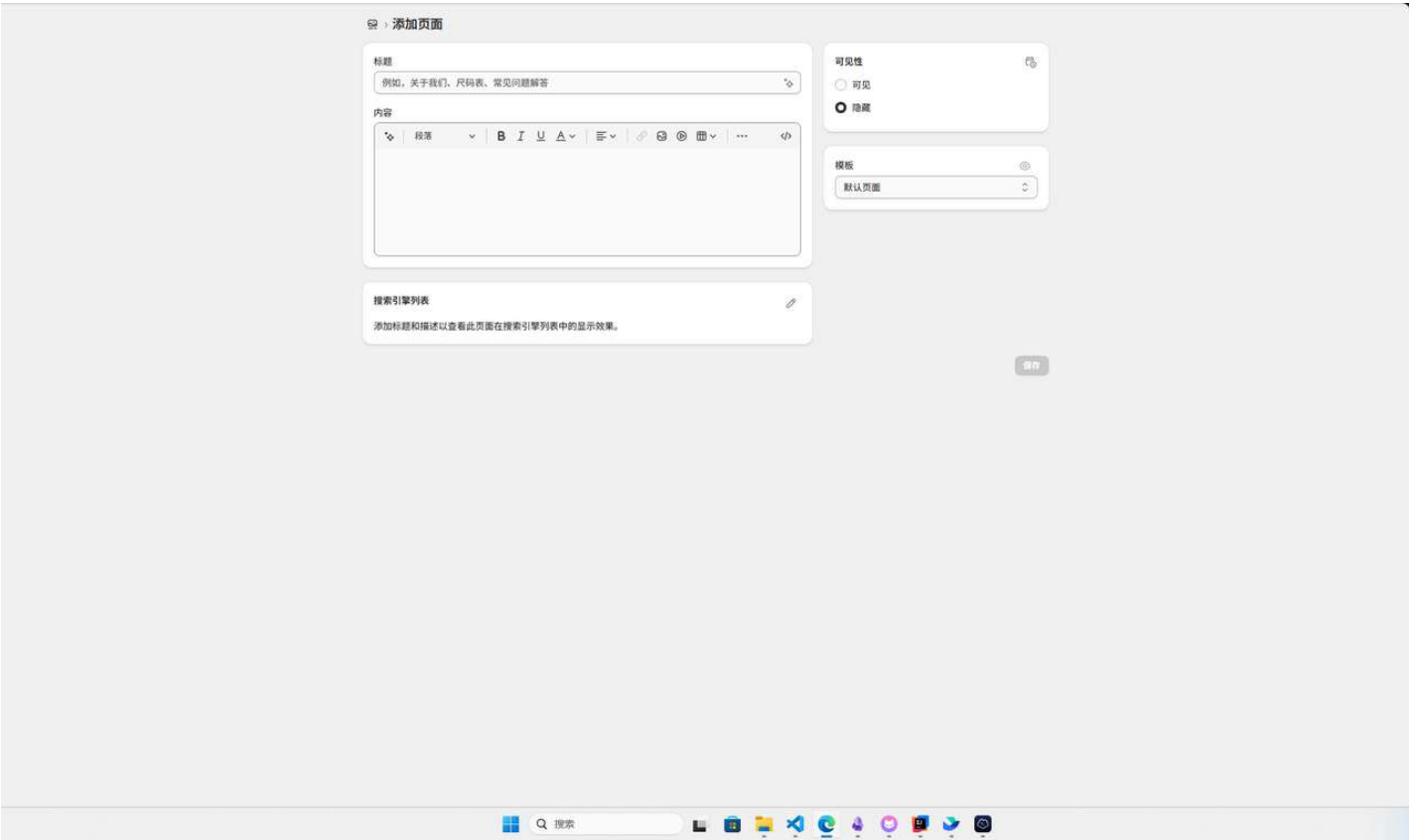
- 关于官方的页面参考文档，[Templates](#)
- 对于我们自定义的页面我说明一下如何创建自定义页面，

关于添加页面

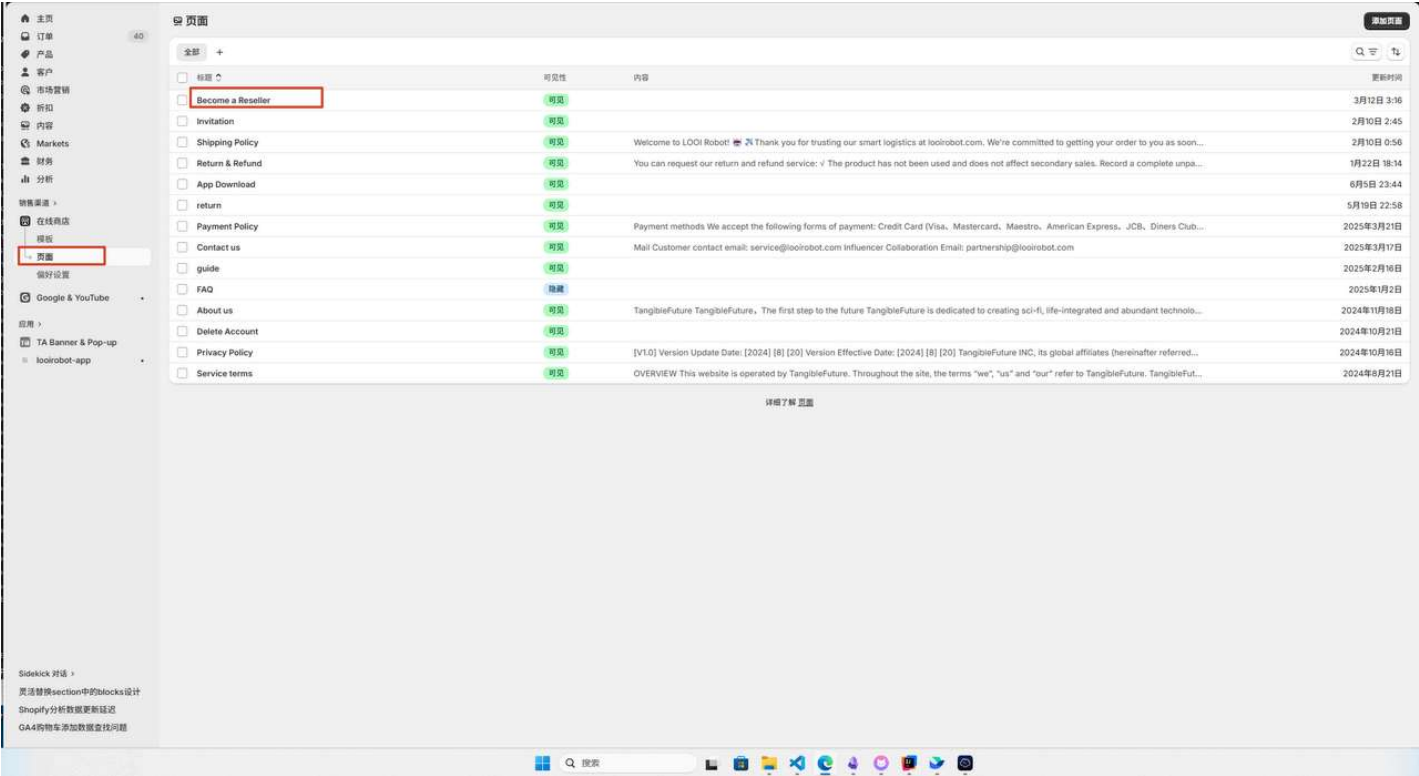
进入



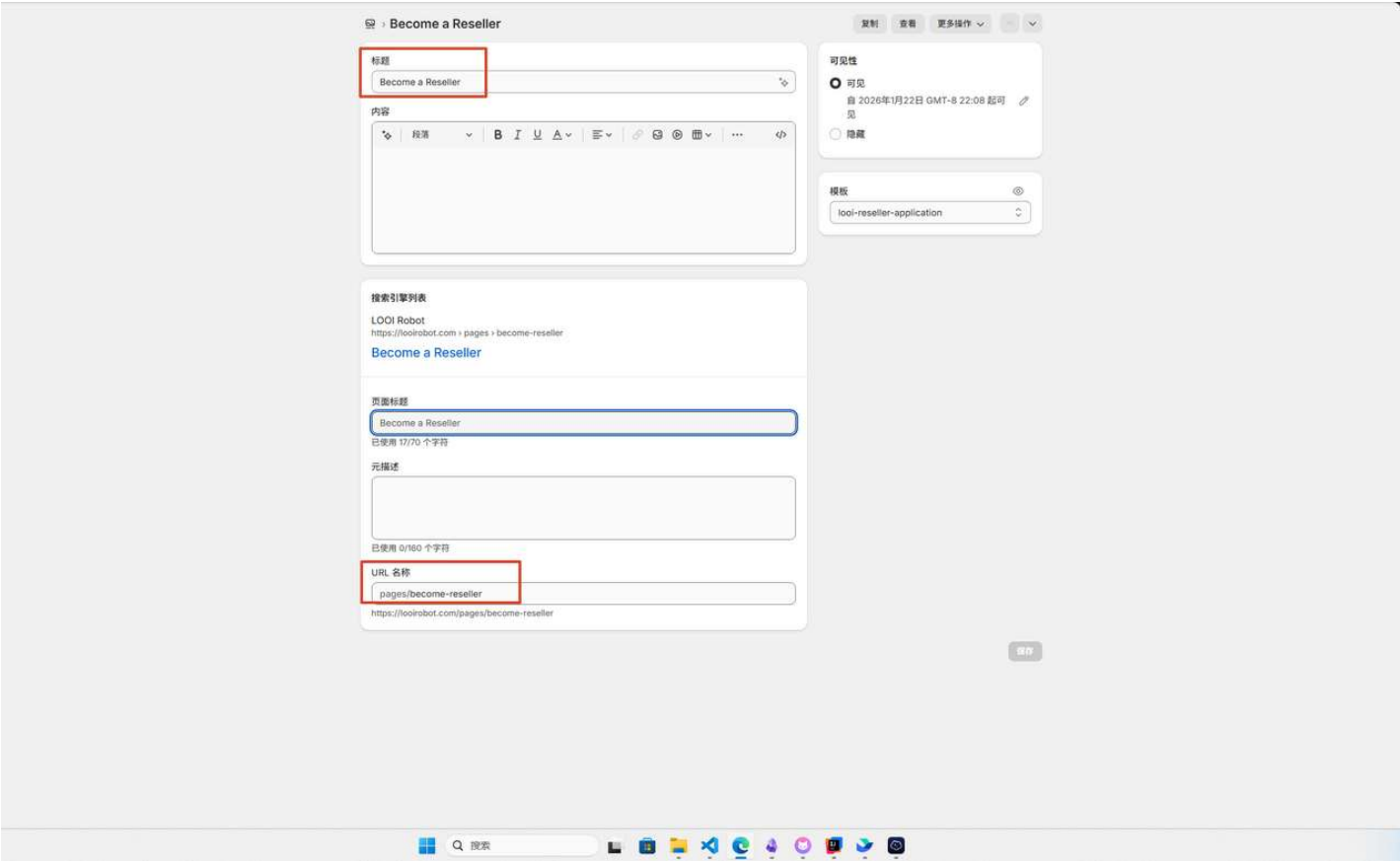
然后点击添加页面，可以看到一些配配置



例如进入如下页面



可以看到我们的经销商问卷的页面，点进去可以配置标题，页面内容以及url



关于标题以及url的使用参见如下

https://looirobot.com/pages/become-reseller

Bonus Offer: Extra \$5 Off Starlight White. →

LOOI Robot Support&Help About Us TANGIBLE FUTURE

LOOI Reseller Application

Company Information ✨

Company Name *

Country/Region *

Company size *

How do you plan to sell LOOI? * (Check all that apply)

☐ Online Store

☐ Physical Retail Stores

☐ Distribution/Wholesale Network

☐ Others

Contact Details 😊

First Name * **Last Name ***

Phone Number *

+

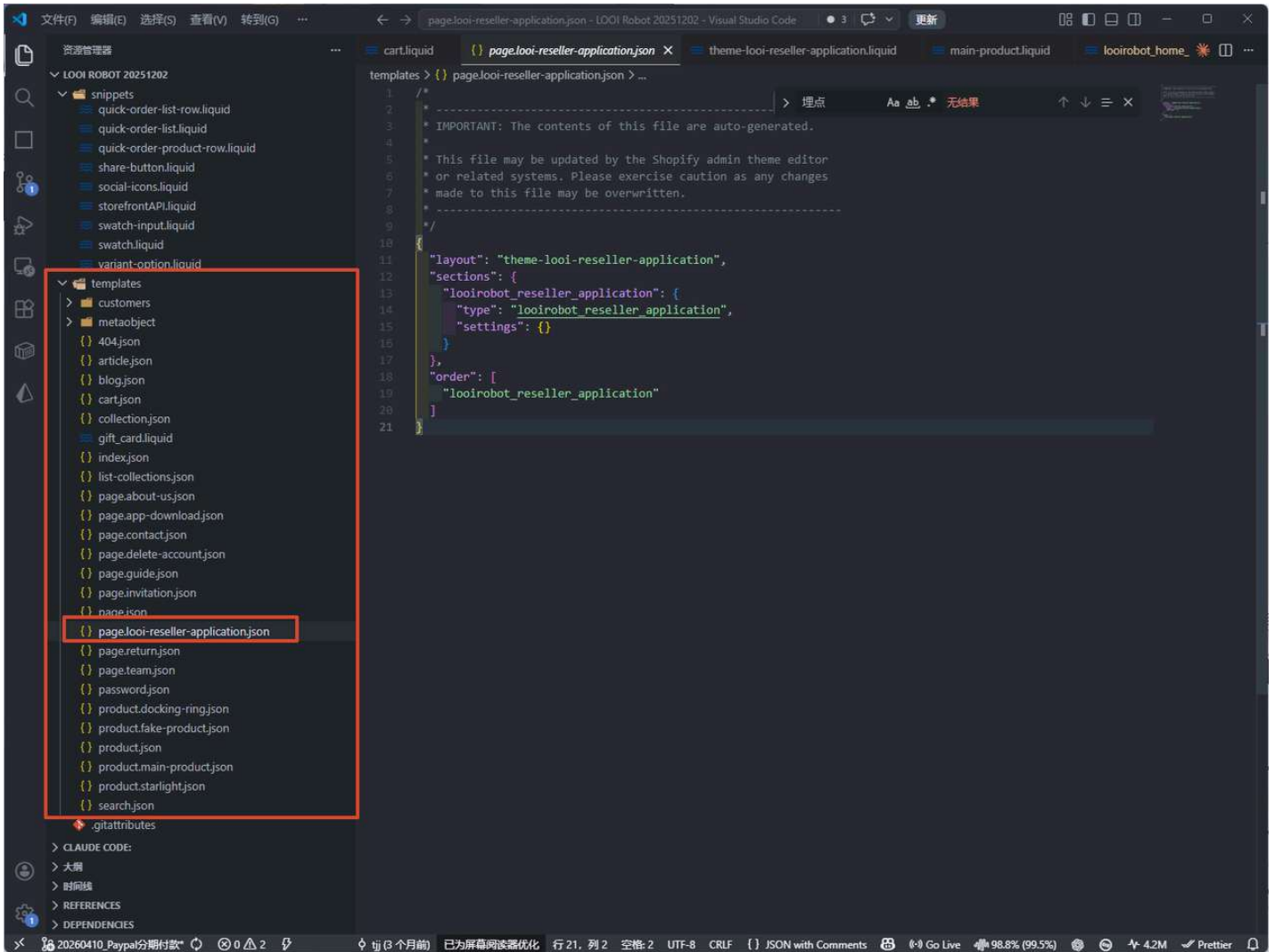
Email *

Job Title

Hey! Got questions? We're here to help you out!

Partnership Proposal

注意添加页面一定要在这里添加，系统会为我们自己生成关于页面的配置文件，这里添加后使用shopify theme pull 拉一下代码就会同步配置
这里就是所有的页面配置



我为你介绍一下关于页面的配置就拿这个经销商页面举第一个例子

代码块

```
1  {
2
3    "layout": "theme-looi-reseller-application", // 使用哪个布局, layout文件夹, 默认是the
4    "sections": {
5      "looirobot_reseller_application": { // 这个section的标签
6        "type": "looirobot_reseller_application", // 使用哪个section文件, 注意要和文件名相
7        "settings": {}
8      }
9    },
10   "order": [
11     "looirobot_reseller_application" // 如果有多个section, 他们之间的顺序
12   ]
13 }
```

我再拿首页举一个例子

代码块

```
1  {
2    "sections": {
3      "main": { // main. 同样是哪个section
4        "type": "main-page", // 使用的是section文件夹里的main-page.liquid
5        "settings": { // 这个是一些配置, 不是有一些内容是提供给市场修改的数据吗, 他的改动就在这
6          "padding_top": 28,
7          "padding_bottom": 28
8        }
9      },
10     "image_banner_TgwyLL": {
11       "type": "image-banner",
```

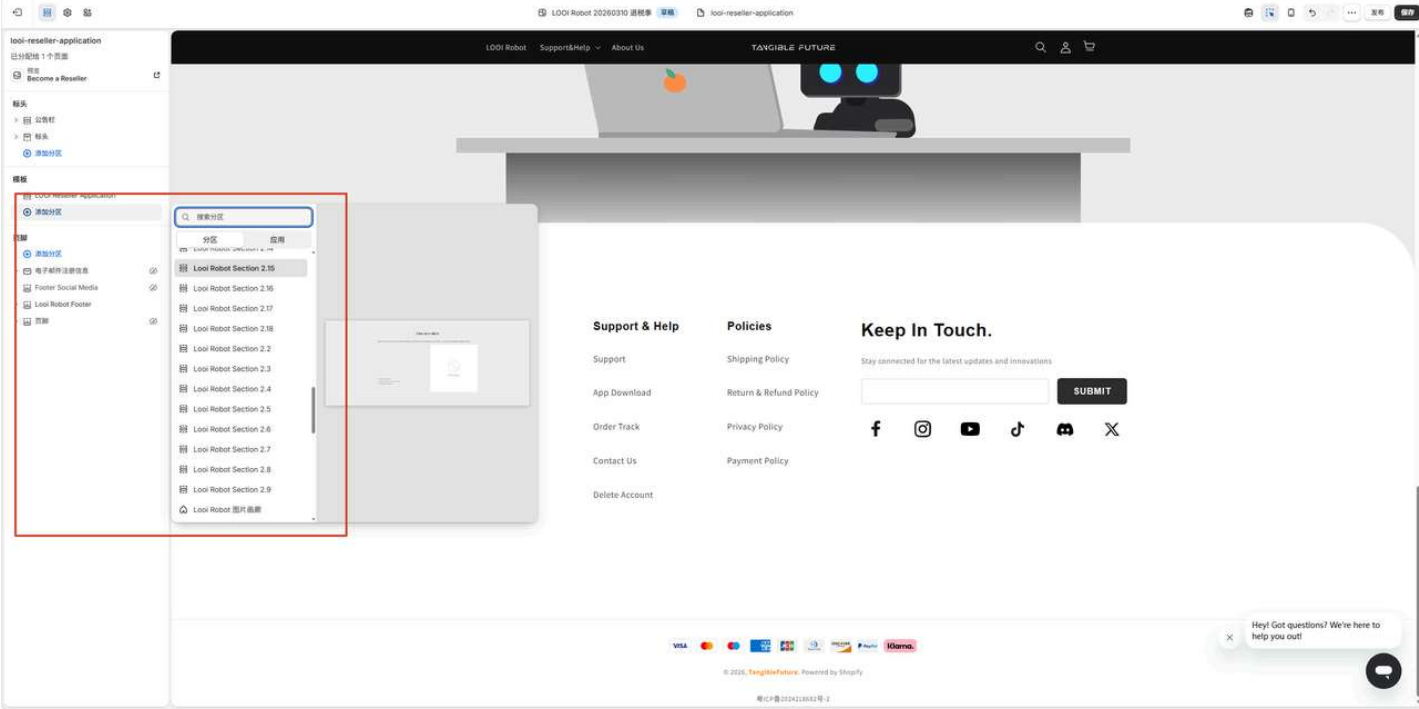
```

12     "blocks": { // 这个是每一个section里面有哪些blocks,section里面的小组件
13         "heading_fcUBYb": {
14             "type": "heading", // 哪个block
15             "disabled": true, // 是否使用
16             "settings": { // 关于一些配置
17                 "heading": "Image banner",
18                 "heading_size": "h1"
19             }
20         },
21         "text_kyUHV7": {
22             "type": "text",
23             "disabled": true,
24             "settings": {
25                 "text": "Give customers details about the banner image(s) or content",
26                 "text_style": "body"
27             }
28         },
29         "buttons_MM4DrG": {
30             "type": "buttons",
31             "disabled": true,
32             "settings": {
33                 "button_label_1": "Button label",
34                 "button_link_1": "",
35                 "button_style_secondary_1": false,
36                 "button_label_2": "Button label",
37                 "button_link_2": "",
38                 "button_style_secondary_2": false
39             }
40         }
41     },
42     "block_order": [// block之间的顺序
43         "heading_fcUBYb",
44         "text_kyUHV7",
45         "buttons_MM4DrG"
46     ],
47     "disabled": true, // 这个section是否可见
48     "settings": { // section的配置
49         "image": "shopify://shop_images/L00I_Robot_About_us_Page.jpg",
50         "image_overlay_opacity": 0,
51         "image_height": "adapt",
52         "desktop_content_position": "middle-center",
53         "show_text_box": true,
54         "desktop_content_alignment": "center",
55         "color_scheme": "scheme-4",
56         "image_behavior": "none",
57         "mobile_content_alignment": "center",
58         "stack_images_on_mobile": true,
59         "show_text_below": true
60     }
61 },
62 "delete_account_k4cAED": { // 另一个section
63     "type": "delete-account",
64     "disabled": true,
65     "settings": {}
66 }
67 },
68 "order": [// 顺序
69     "main",
70     "image_banner_TgwyLL",
71     "delete_account_k4cAED"
72 ]
73 }

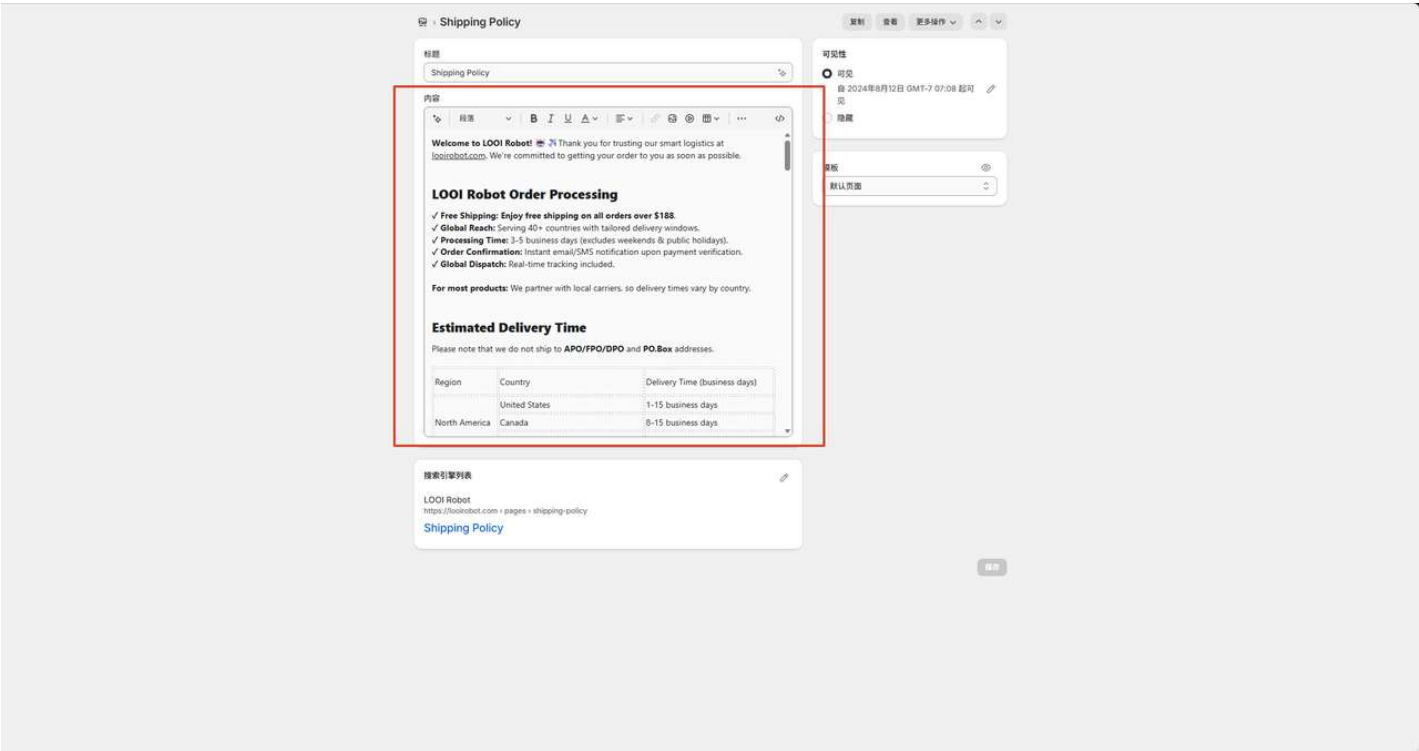
```

!!!! 一半不要手动改上面的配置,除非你完全理解,系统会自己为你配置的,比如你在后台添加界面,比如你为页面添加section块,系统都会为你配置好重点是关于内容,如上所述,你可以为页面添加section,那么我们开发的时候怎么添加呢?

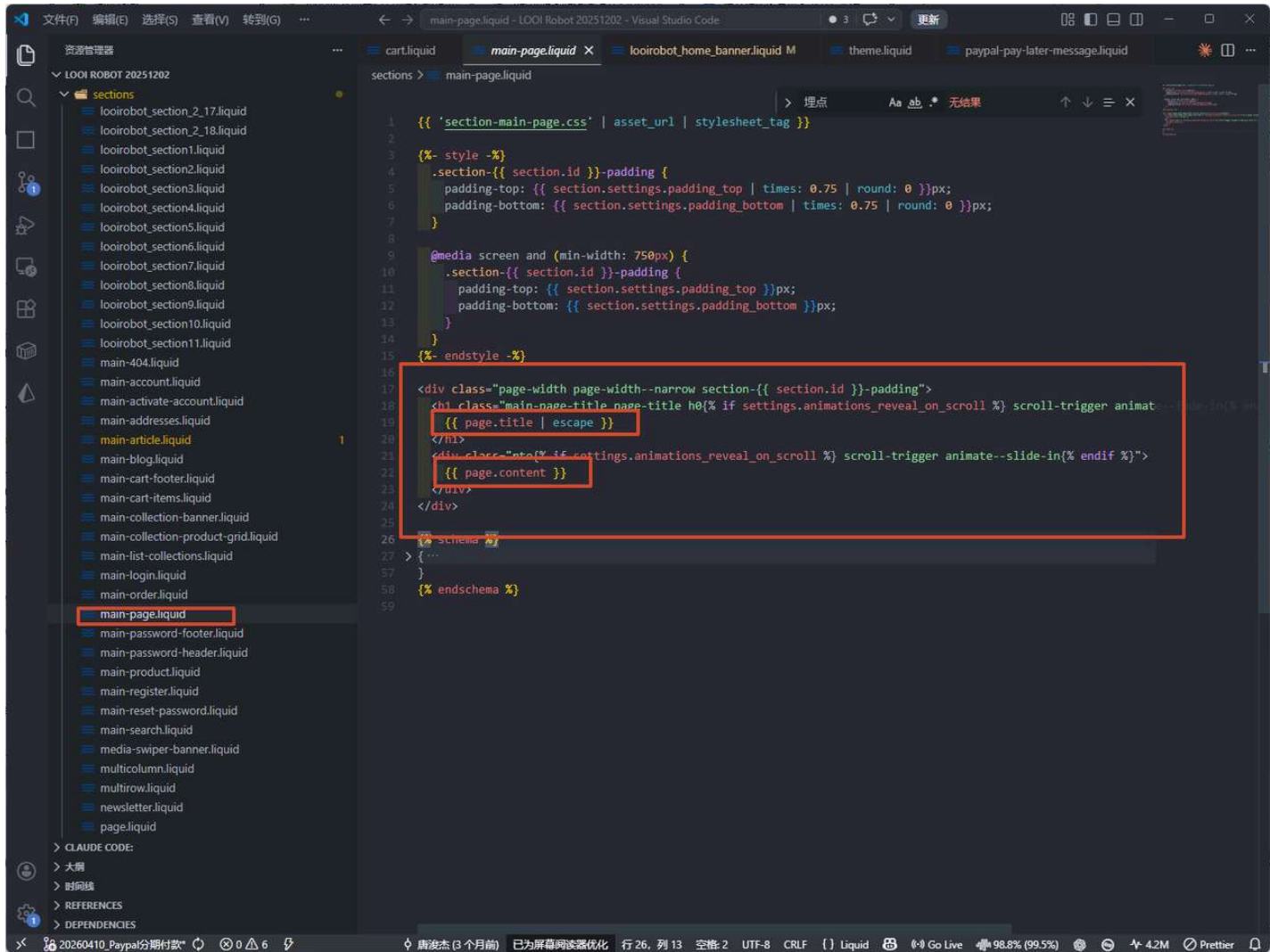
最方便的方式就是首先添加一个**空白的section**,然后推上服务器,只需要包含必要的schema配置,比如当前section块的名字,然后在shopify后台进入模板添加section



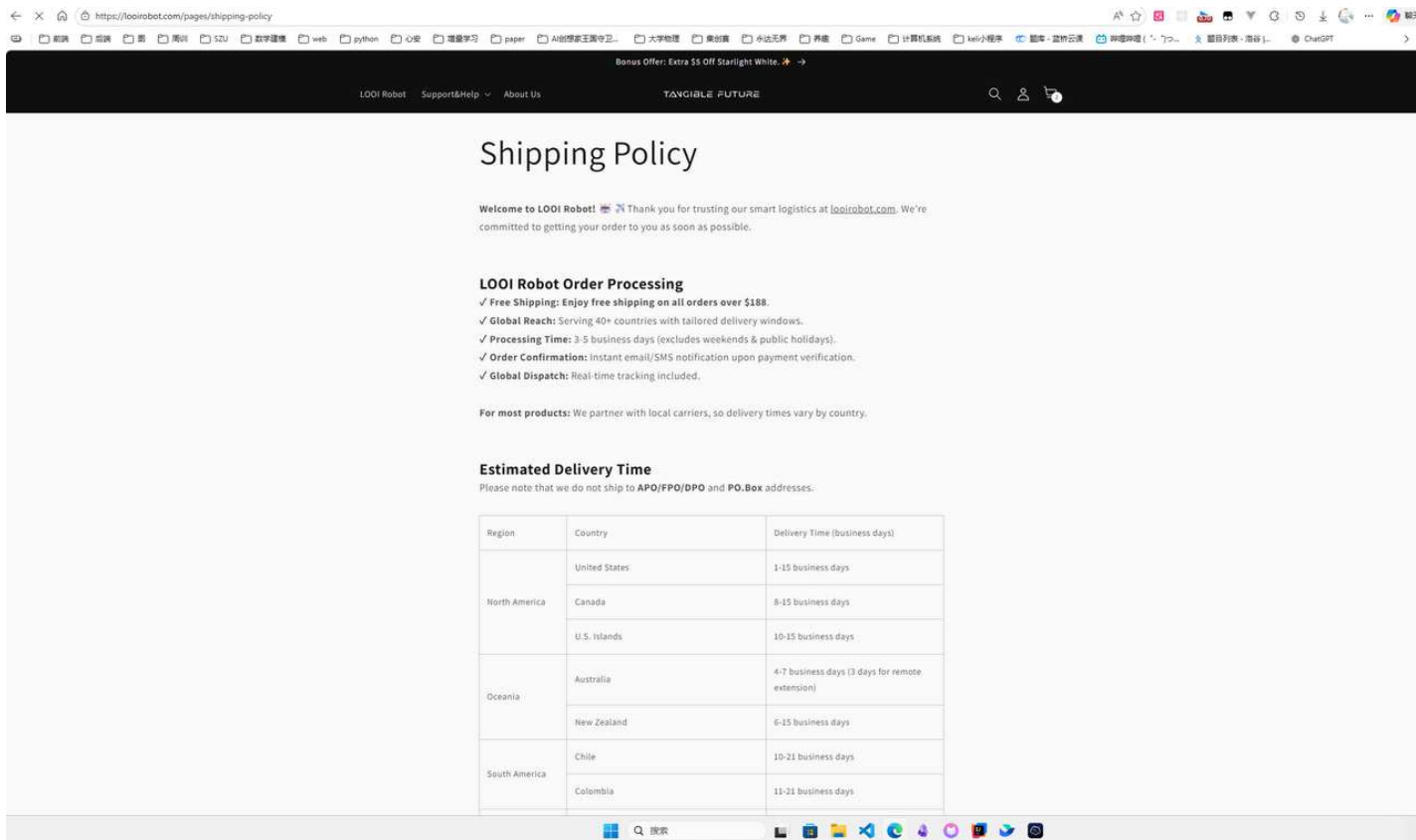
然后系统会为你自动配置,你的页面的json配置,
然后是关于页面内容的编辑



其实内容和标题就渲染在这里



然后页面呢就是这样



如上只是一些简单的介绍全部来自于官方文档,更多详情不再赘述,请仔仔细细查看官方文档,或者询问官方给的ai工具

环境配置

CLI安装

使用shopify CLI开发,查看文档[Shopify Dev Docs](#)

代码块

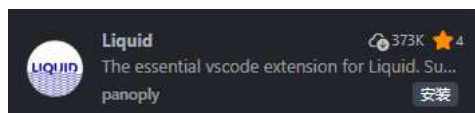
```
1 npm i -g @shopify/cli@latest
```

vscode插件安装

shopify语法提示



liquid语法高亮, 格式化



代码中使用到的技术栈

html,liquid

liquid语法,多了一些官方扩展了html语法,参见文档[Liquid reference](#)

推荐优先使用liquid,如果使用liquid语法的注释是会被用户看到

liquid语法中重要的一部分是提供给市场在独立站后台编辑的参数

详情参见文档,注意这个文档一定要看完,它包含liquid的各种实用的语法标签,节省开发成本

🔍 情人节按钮

...

✕

按钮文字

SHOP NOW

按钮链接

粘贴链接或搜索

PC端设置

按钮宽度 - px

297

按钮高度 - px

59

字体大小 - px

27

距离顶部 - %

30

距离左侧 - %

8

圆角 - px

14

移动端设置

按钮宽度(移动) - px

200

按钮高度(移动) - px

50

字体大小(移动) - px

25

距离顶部(移动) - %

73

```
{% schema %}
{
  "name": "情人节按钮",
  "settings": [
    {
      "type": "text",
      "id": "button_text",
      "label": "按钮文字",
      "default": "SHOP NOW"
    },
    {
      "type": "url",
      "id": "button_link",
      "label": "按钮链接"
    },
    {
      "type": "header",
      "content": "PC端设置"
    },
    {
      "type": "number",
      "id": "button_width",
      "label": "按钮宽度 - px",
      "default": 297
    },
    {
      "type": "number",
      "id": "button_height",
      "label": "按钮高度 - px",
      "default": 59
    },
    {
      "type": "number",
      "id": "button_font_size",
      "label": "字体大小 - px",
      "default": 27
    },
    {
      "type": "number",
      "id": "button_top",
      "label": "距离顶部 - %",
      "default": 30
    },
    {
      "type": "number",
      "id": "button_left",
      "label": "距离左侧 - %",
      "default": 8
    }
  ]
}
```

CSS

注意响应式,这是最重要的,考虑桌面,平板,手机,三端,流畅过度

注意类名,里面每个页面是由section拼合而成的,你写的代码是什么就呈现的什么,完全没有混淆,所以注意自己的类名的书写一定要限定好范围

其中shopify官方自己有给每个section生成独立的id,推荐用这个

```

<style>
.{{ section.id }} {
  --button-height: {{ section.settings.button_height_pc }}px;
  --button-width: {{ section.settings.button_width_pc }}px;
  --button-top: {{ section.settings.button_top_pc }}%;
  --button-left: {{ section.settings.button_left_pc }}%;
  --button-radius: {{ section.settings.button_radius_pc }}px;
  --background-image: url({{ section.settings.background_image_pc | image_url: width: 1920, height:1080 }});

  --button-font-size: {{ section.settings.button_font_size_pc }}px;
  --hw-ratio: 0.5625;

  /* 16:9比例 (height/width = 9/16 = 0.5625) */
  width: 100%;
  height: calc(100vh * var(--hw-ratio));
  position: relative;
  background-image: var(--background-image);
  background-position: left;
  background-repeat: no-repeat;
  background-size: cover;
  overflow: hidden;
}

.{{ section.id }}
.love's_button {
  width: var(--button-width);
  height: var(--button-height);
  line-height: var(--button-height);
  position: absolute;
  top: var(--button-top);
  left: var(--button-left);
  transform: translate(0, 0);
  font-size: var(--button-font-size);
  cursor: pointer;
  border-radius: var(--button-radius);
  border: none;
  outline: none;
}

```

同时如果有需要提供给市场他们可以调整的参数,务必使用css类名,方便管理也方便做响应式

```

@media screen and (max-width: 750px) {
.{{ section.id }} {
  --button-width: {{ section.settings.button_width_mobile | default: 223 }}px;
  --button-height: {{ section.settings.button_height_mobile | default: 57 }}px;
  --background-image: url({{ section.settings.background_image_mobile | image_url: width: 1800, height:3200 }});
  --button-top: {{ section.settings.button_top_mobile | default: 50 }}%;
  --button-left: {{ section.settings.button_left_mobile | default: 50 }}%;
  --button-radius: {{ section.settings.button_radius_mobile | default: 26 }}px;
  --hw-ratio: 1.77727272727273;
  /* 440:782比例 (height/width = 782/440 = 1.7773) */
  --button-font-size: {{ section.settings.button_font_size_mobile | default: 24 }}px;
}
}

```

JavaScript

alpine.js

额外安装的库,位置assets\cdn.min.js,

已经配置好声明在了全局,声明位置layout\theme.liquid ,

官方文档[Alpine.js](#)

可以看作是一个轻量化的vue,主要为了简化开发,极简的实现页面数据响应式

对于配置的封装放在这里面snippets\storefrontAPI.liquid

这是使用的样例可以参考snippets\cart.liquid

storefrontAPI

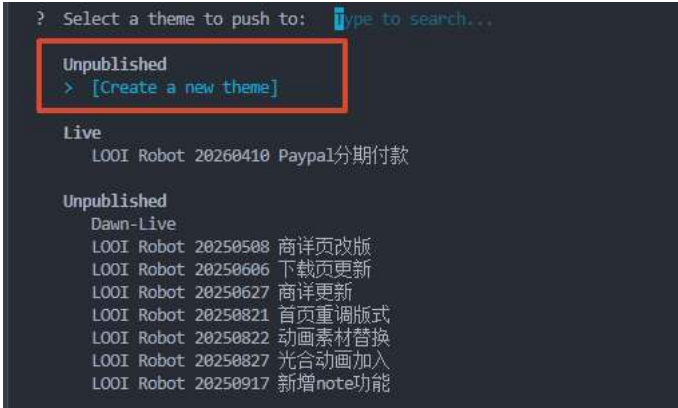
官方文档[Storefront API reference](#)

在 Shopify 里, **Storefront API** 是用来构建“自定义前台 (Custom Storefront)”的 GraphQL API

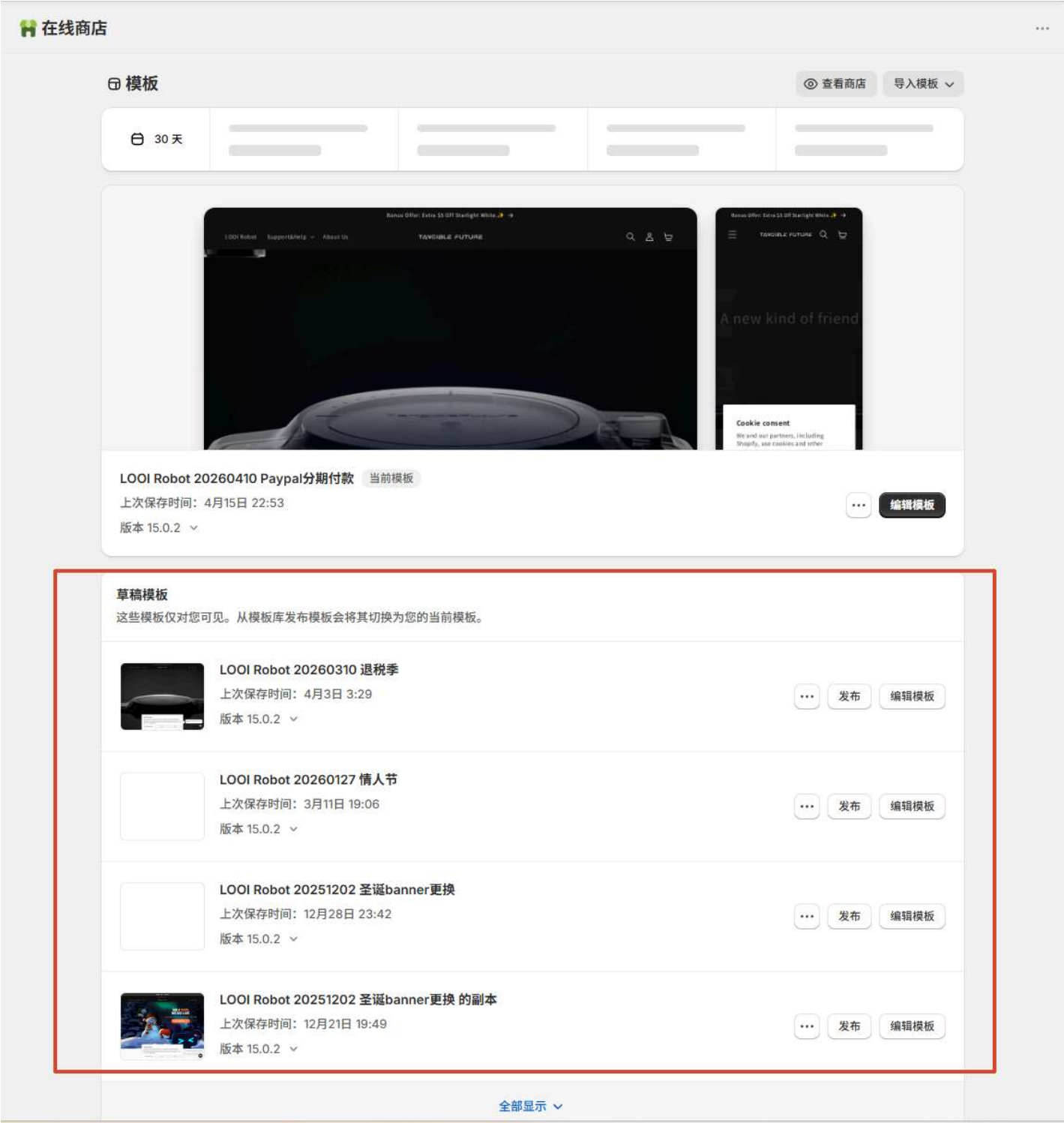
“一个专门给买家端 (前台) 用的接口层, 用来查商品、集合、购物车、结账等一切买家看到和操作的東西。”

开发规范

1. 没有完全的把握之前,不要动旧的代码
2. main分支始终要同步线上的live的页面每次开发前使用shopify theme pull拉取云端代码,然后提交git,提交信息里最好标识,干了什么,以及拉取的时间
3. 每次开发新功能都git另起一个分支,同时使用shopify的CLI工具创建一个新的theme,



另起的theme会出现在这里



然后市场通过审核后,他们自己上线

4. 我们开发新功能一定要在live基础之上开发,如果要上线live,注意把分支代码合并到main分支,因为有时候有些小的需求可能直接就在main上直接修改了,比如



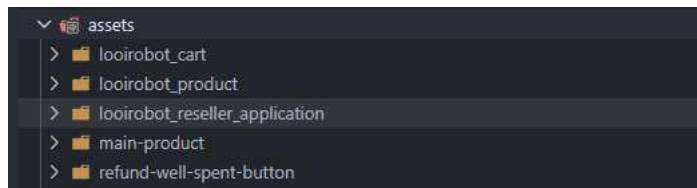
5. 不要回退到以前的旧的版本,可能有些bug改了,那么什么时候需要回退旧版本呢,如果当前的功能有问题,那么正好可以回退版本,之前开发新功能有建立分支,正好直接回退到直接可用的版本
6. 尽量不要把函数声明到全局,就算要声明到全局也使用一个类把函数包起来比如

代码块

```
1  /**
2   * Storefront API 请求类
3   */
4  class StorefrontAPI {
5      constructor(config) {
6          this.config = config;
7      }
8      /**
9       * 执行 GraphQL 查询或 mutation
10     *
11     * @param {string} query - GraphQL 查询字符串
12     * @param {Object} variables - GraphQL 变量对象
13     * @returns {Promise<Object>} API 响应数据
14     * @throws {Error} 当请求失败或返回错误时抛出异常
15     */
16     async query(query, variables = {}) {
17         try {
18             const response = await fetch(this.config.apiUrl, {
19                 method: 'POST',
20                 headers: {
21                     'Content-Type': 'application/json',
22                     'X-Shopify-Storefront-Access-Token': this.config.accessToken,
23                 },
24                 body: JSON.stringify({
25                     query,
26                     variables
27                 })
28             });
29             // 检查 HTTP 响应状态
30             if (!response.ok) {
31                 throw new Error(`HTTP Error: ${response.status} ${response.statusText}`);
32             }
33             const result = await response.json();
```

```
34      // 检查 GraphQL 错误
35      if (result.errors && result.errors.length > 0) {
36          console.error('GraphQL Errors:', result.errors);
37          throw new Error(`GraphQL Error: ${result.errors[0].message}`);
38      }
39      return result;
40  } catch (error) {
41      console.error('Storefront API Request Failed:', {
42          query: query.slice(0, 100) + '...', // 只显示前100字符
43          variables,
44          error: error.message
45      });
46      throw error;
47  }
48  }
49  }
50
51  // ===== 创建全局实例 =====
52  window.StorefrontAPI = new StorefrontAPI(STOREFRONT_CONFIG);
```

7. 对于一些页面UI需要使用到的静态资源注意用一个文件夹包裹,方便管理



但是对于其他文件, 例如sections,不能建文件夹,必须平铺!

8. 对于sections的文件命名,加上前缀looirobot便于区分后来新开发的文件,后缀标识文件功能,语义化更明显一点,方便后续管理,拒绝这样的命名"looirobot_section_2_1.liquid"!
9. 对于独立站后台,功能能看不能动,特别是用户数据,动之前问问市场
10. 如果后续有什么需要后端支持的东西,那你自己搭建一个后端,注意国内外都要能访问, 注意要用https的请求, 还有前端代码不会做混淆, 不要放敏感信息
11. 关于代码规范, 以及响应式的技巧, 可以查看我已经书写的代码, git里提交的代码都是我书写的, 强调, 一定注意提交, 一定注意版本管理, 一定注意代码规范

目前需要注意的业务

[独立站优化专项 - 飞书云文档](#)

一些需求文档参见

独立站优化专项

一、商详 (2506-已完成)
二、首页 (2507-已完成)
三、网页弹窗&倒计时插件 (2512-已完成)
四、鼠标跟踪动画 (2512-已完成)
五、Reseller页面 (2501-已完成)
六、支付页面优化 (2501-已完成)
七、首页&商详页 (2503-推进中)

注: 文案对应图片 黄菲: 当前视觉仅作参考, UI&UX属性对齐apple、DJI、insta360等, 移动端和PC端同步优化 (移动端为重点) 李弘立

1. 首页

独立站优化专项

一、商详 (2506-已完成)
二、首页 (2507-已完成)
三、网页弹窗&倒计时插件 (2512-已完成)
四、鼠标跟踪动画 (2512-已完成)
五、Reseller页面 (2501-已完成)
六、支付页面优化 (2501-已完成)
七、首页&商详页 (2503-推进中)

注: 文案对应图片 黄菲: 当前视觉仅作参考, UI&UX属性对齐apple、DJI、insta360等, 移动端和PC端同步优化 (移动端为重点) 李弘立

1. 首页

模块	视觉	文案	UI&UX参考 (仅作参考, 可根据更多参考自行调整)
主视觉			
引言		Technology was never meant to serve productivity alone. LOOI — A new kind of friend. Infusing machines with soul, redefining	

12. 查看网站域名

LOOI Robot

looirobot.com

Search

General
Plan
Billing
Users
Payments
Checkout
Customer accounts
Shipping and delivery
Taxes and duties
Locations
Markets
Apps
Sales channels
Domains
Customer events
Notifications
Metafields and metaobjects
Languages
Customer privacy
Policies

立强 曾
jluxiaotu@gmail.com

Domains

Connect... Buy new domain

Domain	Status
looirobot.com	Primary Connected
f10c7c-17.myshopify.com	Connected
www.looirobot.com	Connected
tangiblefuturelab.com	Invalid DNS
www.tangiblefuturelab.com	Connected

Learn more about domains

Home
Orders 42
Products
Customers
Marketing
Discounts
Content
Markets
Finance
Analytics
Sales channels >
Online Store
Apps >

Settings

Last 30 days All channels 1 live visitor

Sessions 41.8K > 4% Total sales \$53.8K > 10% Orders 264 > 1.1% Conversion rate 0.53% > 2%

16 orders to fulfill

Good evening, let's get started.

Ask anything...

You're now selling directly in Microsoft Copilot
Your products are now available to purchase in Microsoft Copilot through Agentic Storefronts. Reach millions of buyers where they search. No additional fees.
Manage Agentic Storefront Learn more

Offer: Try the Plus plan at your current monthly platform fee
Get access to two months of Plus-exclusive features like unlimited B2B catalogs, Audiences, and priority support at your current monthly platform fee.
Start 2-month trial

Upgrade to the new abandoned checkout automation

经销商问卷

第一个就是我们的经销商问卷, 页面在Become a Reseller – LOOI Robot 代码在sections\looirobot_reseller_application.liquid 里面有两个服务,一个是文件的上传

https://f5d23thc7h.feishu.cn/wiki/OVjiw5zoMiee5KkJfDcZfiwnqd

17/35

```
// 单文件上传函数,
const uploadOneFile = ({ file, item, progressBar, errorMsgDom } = {}) => {
  const xhr = new XMLHttpRequest();
  const formData = new FormData();
  formData.append('file', file);
  // required param
  formData.append('type', '1');

  xhr.open('POST', 'https://looirobot-app.vercel.app/api/public/upload');

  // 进度处理
  xhr.upload.onprogress = (ev) => {
    if (ev.lengthComputable) {
      const percent = Math.round((ev.loaded / ev.total) * 100);
      if (progressBar) progressBar.style.width = percent + '%';
      // 当字节上传完成但服务器仍在处理时, 显示 processing 状态
      if (percent === 100) {
        item.classList.add('processing');
        item.classList.remove('uploading');
        if (errorMsgDom) errorMsgDom.textContent = 'Processing...';
      }
    }
  };
};
```

另一个是表单的提交

```
}

const formValues = getFormValues();
// 表单提交
fetch('https://looirobot-app.vercel.app/api/public/reseller-applications', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(formValues),
})
.then((res) => res.json())
.then((json) => {
  console.log('submit response', json);
});
```

注意这两个的维护, 后续可以考虑迁移到我们新的独立站后台

涉及购物车, 订单支付方面的

我们的市场需要分析独立站的各种销售数据, 用户的转化率等等, 数据都来自埋点,

如果我们的支付, 或者添加购物车接口使用storefrontAPI, 要注意这个是不会自动上传埋点数据的, 需要我们手动追踪, 发送数据

```
/**
 * 手动触发 GA4 追踪（私有方法）
 *
 * @param {Object} cart - 购物车对象
 * @param {string} merchandiseId - 刚添加的商品变体 ID
 * @param {number} quantity - 添加的数量
 * @private
 */
static _trackAddToCart(cart, merchandiseId, quantity) {
  try {
    // 从购物车中找到刚添加的商品行
    const addedLine = cart.lines.edges.find(
      ({ node }) => node.merchandise.id === merchandiseId
   )?.node;

    if (!addedLine) return;

    const merchandise = addedLine.merchandise;
    const price = parseFloat(merchandise.price.amount);
    const currency = merchandise.price.currencyCode;

    if (typeof gtag === 'function') {
      const eventData = {
        currency: currency,
        value: price * quantity,
        items: [{
          item_id: CartManager.getNumericId(merchandise.id),
          item_name: merchandise.product.title,
          item_variant: merchandise.title,
          price: price,
          quantity: quantity
        }]
      };
      gtag('event', 'add_to_cart', eventData);
      console.log('GA4 add_to_cart 事件已触发', eventData);
    } else {
      console.warn('gtag 未加载, GA4 add_to_cart 事件未触发');
    }
  } catch (e) {
    console.warn('GA4 追踪失败:', e);
  }
}
```

但是目前有问题埋点不可用，所以我们回退了代码使用了这种api

```
/**
 * 商品API请求封装
 */
2 个引用
class ProductAPI {
  /**
   * 获取商品详情
   * @param {string} handle - 商品handle
   * @returns {Promise} 商品详情数据
   */
  0 个引用
  static async getProductDetails(handle) {
    try {
      const response = await fetch(`/products/${handle}.js`);
      if (!response.ok) throw new Error("获取商品详情失败");
      return await response.json();
    } catch (error) {
      console.error("获取商品详情出错:", error);
      throw error;
    }
  }

  /**
   * 增加购物车商品数量
   * @param {number} variantId - 商品变体ID
   * @param {number} quantity - 要增加的数量
   * @returns {Promise} 更新后的购物车数据
   */
  0 个引用
  static async addToCart(variantId, quantity = 1) {
    try {
      const response = await fetch("/cart/add.js", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          items: [
            {
              id: variantId,
              quantity: quantity,
            },
          ],
        }),
      });

      if (!response.ok) throw new Error("添加商品失败");
      return await response.json();
    } catch (error) {
      console.error("添加商品出错:", error);
      throw error;
    }
  }
}
```

他会自动上传埋点数据的，这个需要注意

LOOI Admin 项目

一、项目概述

1.1 项目简介

LOOI Admin 是一个企业内部管理系统，用于管理 LOOI 机器人平台的应用配置、行为编辑、国际化等核心功能。

这个项目重点在于使用ai,毕竟是一个管理后台,注意一定要看一遍ai的代码,对于旧的功能迁移,强调千万不要新建数据库,可能会影响到旧的功能的使用,还要注意代码的模块化,封装解耦,这样方便ai自己读取上下文,也方便我们自己管理

项目地址: <https://gitee.com/tangiblefuture/looi-admin.git>

1.2 技术栈

后端 (looi-admin-server)

技术	版本	说明
Spring Boot	3.2.5	核心框架
MyBatis Plus	3.5.6	ORM 框架
MySQL	8.0+	数据库
Redis	6.0+	缓存
JWT	0.12.6	认证令牌
Knife4j	4.5.0	API 文档

前端 (looi-admin-web)

技术	版本	说明
Vue	3.4.21	核心框架
Vite	5.1.6	构建工具
Element Plus	2.6.1	UI 组件库
TypeScript	6.0.2	类型系统
Pinia	2.1.7	状态管理
D3.js	7.9.0	时间轴可视化
libpag	4.5.42	PAG 动画

二、项目结构

代码块

```
1  looi-admin/  
2  |—— .claude/                # Claude Code 配置  
3  |  |—— agents/              # Agent 配置  
4  |  |—— commands/           # 自定义命令  
5  |  |—— rules/              # 编码规范  
6  |  |  |—— coding-style.md  
7  |  |  |—— frontend-style.md  
8  |  |  |—— security.md  
9  |  |  |—— testing.md  
10 |  |—— skills/              # 技能配置  
11 |—— docs/                  # 项目文档  
12 |—— looi-admin-server/     # 后端服务 (Spring Boot)  
13 |—— looi-admin-web/        # 前端管理 (Vue3)  
14 |  |—— SourceCode/         # 老项目源码 [待重构]  
15 |    |—— looi--web-action-design  
16 |    |—— looirobot-back-end-management
```

2.1 后端目录结构

代码块

```
1  looi-admin-server/  
2  |—— src/main/java/com/looi/server/  
3  |  |—— LooIAdminApplication.java  # 启动类  
4  |  |—— config/                   # 配置类  
5  |  |  |—— SecurityConfig.java    # 安全配置  
6  |  |  |—— CorsConfig.java       # 跨域配置  
7  |  |—— modules/                 # 功能模块  
8  |  |  |—— iam/                   # 身份认证模块  
9  |  |  |—— i18n/                  # 国际化模块  
10 |  |  |—— app/                   # 应用管理模块  
11 |  |  |—— common/               # 公共组件  
12 |  |  |—— result/               # 统一响应  
13 |  |  |—— exception/            # 异常处理  
14 |  |  |—— util/                 # 工具类  
15 |—— src/main/resources/  
16 |  |—— application.yml           # 主配置  
17 |  |—— application-dev.yml      # 开发环境配置
```

```
18 | application-prod.yml | # 生产环境配置
```

2.2 前端目录结构

```
代码块
1 | looi-admin-web/
2 | | src/
3 | | | api/ | # API 接口
4 | | | | modules/ | # 分模块 API
5 | | | | components/ | # 公共组件
6 | | | | | ActionCanvasPanel.vue | # 动作画布面板
7 | | | | | PagPreview.vue | # PAG 动画预览
8 | | | | | resource/ | # 资源相关组件
9 | | | | composables/ | # 组合式函数
10 | | | | | useBehaviorTimeline.ts | # 时间轴逻辑
11 | | | | | useBehaviorStatus.ts | # 状态管理
12 | | | | views/ | # 页面视图
13 | | | | | app/ | # APP 模块页面
14 | | | | | | behavior/ | # 行为管理
15 | | | | | | | action/ | # 动作序列
16 | | | | | | iam/ | # IAM 模块页面
17 | | | | | | i18n/ | # 国际化页面
18 | | | | stores/ | # Pinia 状态管理
19 | | | | | utils/ | # 工具函数
20 | | | | | router/ | # 路由配置
21 | | | .env | # 开发环境变量
22 | | | .env.production | # 生产环境变量
23 | | vite.config.ts | # Vite 配置
```

三、模块说明

3.1 IAM 模块 (身份认证)

包路径: com.looi.server.modules.iam

功能: 用户管理、角色管理、菜单管理、字典管理

核心表:

- sys_user - 系统用户
- sys_role - 角色
- sys_menu - 菜单/权限
- sys_dict / sys_dict_item - 字典

3.2 I18N 模块 (国际化)

包路径: com.looi.server.modules.i18n

功能: 多语言支持、文本管理、导入导出

核心表:

- i18n_language - 语言
- i18n_text - 文本
- i18n_text_value - 文本值

3.3 APP 模块 (应用管理)

包路径: com.looi.server.modules.app

功能: 应用管理、动作序列、动作配置、表情交互、行为管理

核心表:

- app_application - 应用
- app_menu - 多级菜单
- action_sequence - 动作序列

- `action_config` - 动作配置
- `emotion_emotion` - 表情
- `behavior_behavior` - 行为
- `behavior_group` - 分段行为

四、核心功能详解

4.1 行为编辑器

行为编辑器是项目核心功能，支持时间轴可视化编辑。

文件位置: `looi-admin-web/src/views/app/behavior/BehaviorEdit.vue`

关键依赖:

- `src/composables/useBehaviorTimeline.ts` - 时间轴逻辑
- `src/composables/useBehaviorStatus.ts` - 状态管理
- `src/utils/timelineConverter.ts` - 数据格式转换

4.1.1 功能列表

功能	说明	优先级
时间轴渲染	11 轨道 D3.js 可视化	P0
播放控制	播放/暂停/停止	P0
时间块拖拽	拖拽调整时间位置	P0
时间块删除	右键删除	P0
轨道点击添加	点击轨道添加默认块	P0
选中高亮	点击选中并高亮	P0
时间轴缩放	滚轮缩放	P1
跨轨道框选	Ctrl+框选多轨道素材	P1

4.1.2 11 个轨道定义

轨道 Key	名称	数据字段	颜色	可右拉	可左拉
emotion	表情	animationPath	#61E5FA	✓	✓
emotionAction	表情交互	actionCode	#ADA3D3	✓	X
pendant	挂件	animationPath	#76ADFF	✓	✓
action	动作	robotAction	#D36C56	✓	X
audio	音效	voice	#ADA346	✓	✓
mouth	嘴巴	mouth	#ADA3D3	✓	✓
shake	震动	shakePath	#A14F81	X	X
alert	弹窗	-	#4177A9	✓	X
leisure	空闲	-	#60E6A6	✓	X
instant	瞬时	-	#D71313	X	X
content	内容展示	contentDisplay	#60E6A6	✓	X

4.1.3 拉伸规则

右拉 (持续时间延展) : 除 shake、instant 外均可

左拉 (资源裁剪) : 仅 emotion、pendant、audio、mouth 且需满足 `maxTimeSec` 存在

4.1.4 数据存储

行为时间轴数据存储在 `sys_behavior_operation_record.actionContent` 字段, 使用老系统 JSON 格式:

代码块

```
1  [  
2    {
```

```
3      "performerNumber": "02",
4      "data": { "robotAction": "A001" },
5      "startTime": 0,
6      "endTime": 0,
7      "startProgress": 0,
8      "endProgress": 1
9    }
10  ]
```

performerNumber 轨道映射:

performerNumber	轨道
0	瞬时 (instant)
1	表情 (emotion)
2	动作 (action)
3	震动 (shake)
4	弹窗 (alert)
5	空闲 (leisure)
6	音效 (audio)
7	内容展示 (content)
8	挂件 (pendant)
9	嘴巴 (mouth)
10	表情交互 (emotionAction)

4.2 动作序列管理

文件位置: looi-admin-web/src/views/app/action/ActionEditor.vue

核心能力:

- 画布式编辑动作序列
- 拖拽调整位置
- 连线表示运动通道
- 试运行预览

数据结构 (16 秒 / 256 帧 @ 16fps) :

通道	类型值	数据范围
Velocity 前后	1	-100 ~ 100
Direction 左右	2	-100 ~ 100
Pitch 俯仰	3	0 ~ 60°
预留通道	4	-

动作删除前检查:

- 调用 GET /admin/app/actions/{id}/references 检查被行为引用
- 有引用时弹出详情, 不可直接删除

4.3 国际化管理

文件位置: looi-admin-web/src/views/i18n/

4.3.1 功能列表

功能	说明
语言管理	增删改查支持的语言
文本管理	键值对管理多语言文本
导入	支持 JSON/Properties/XML 格式导入
导出	支持 JSON/Properties/XML 格式导出
批量操作	批量导入导出

4.3.2 导入导出格式

格式	文件扩展名	用途
JSON	.json	前端多语言
Properties	.properties	Android/Java
XML	.xml	iOS

4.4 行为审核

相关文件：

- `looi-admin-web/src/views/app/behavior/` 行为列表、待审核列表
- `looi-admin-server/modules/app/` 后端审核 API

4.4.1 状态定义

status	名称	说明
1	设计中	正在编辑
2	待定	暂停
3	延期处理	延后
4	已完成	可提交审核
5	待验收	等待验收
6	需要修改	审核驳回
8	已发布	正式上线

4.4.2 类型定义

type	名称	说明
0	行为元素	正常行为
1	分段行为	分段组合
2	测试态	仅测试使用

4.4.3 审核流程

代码块

```
1  设计中(1) → 已完成(4) → 待验收(5) → 已发布(8)
2      ↓           ↓
3  type=2      审核驳回(6) → 返回设计中(1)
4  (测试态)
```

4.4.4 API 接口

接口	方法	说明
/admin/app/behaviors/pending	GET	获取待审核列表 (status=4)
/admin/app/behaviors/{id}/test	POST	设为测试态
/admin/app/behaviors/{id}/test	DELETE	退出测试态
/admin/app/behaviors/audit	POST	审核行为

4.5 行为组同步

功能：将行为组同步到海外/国内端

API: POST /admin/app/behavior-groups/sync

同步逻辑：

- 根据 type 参数调用不同端点（0=CN, 1=EN）
- 同步行为组、元素列表、元素详情
- 成功后更新本地 syncState / syncStateCn 状态

前端界面：

- 表格显示"同步状态"列
- 操作列有"海外同步"/"国内同步"按钮
- 已同步时按钮禁用

4.6 资源管理

文件位置：looi-admin-web/src/views/app/resource/

4.6.1 8 种资源类型

资源类型	API 路径	说明
表情	/admin/app/resources/emotions/page	animationPath
挂件	/admin/app/resources/pendants/page	animationPath
音效	/admin/app/resources/audios/page	voice
贴图	/admin/app/resources/textures/page	texturePath
特效	/admin/app/resources/effects/page	effectPath
嘴巴	/admin/app/resources/mouths/page	mouth
TTS	/admin/app/resources/tts/page	ttsPath
瞬时	/admin/app/resources/instants/page	instantPath

4.6.2 列表页功能

- 分页查询
- 搜索筛选
- 状态管理

五、API 接口

5.1 认证接口

代码块

```
1  POST /admin/auth/login      - 登录
2  POST /admin/auth/logout    - 登出
```

5.2 IAM 接口

代码块

```
1  GET  /admin/user/info      - 获取当前用户信息
2  GET  /admin/iam/users      - 用户列表
3  POST /admin/iam/users      - 创建用户
```

5.3 APP 接口

代码块

```
1  # 动作序列
2  GET  /admin/app/actions      - 动作列表
3  POST /admin/app/actions      - 新增动作
4  PUT  /admin/app/actions/{id} - 编辑动作
5  DELETE /admin/app/actions/{id} - 删除动作
6
7  # 行为管理
8  GET  /admin/app/behaviors    - 行为列表
9  GET  /admin/app/behaviors/pending - 待审核列表
10 POST /admin/app/behaviors/audit - 审核行为
11
12 # 行为组
13 GET  /admin/app/behavior-groups - 行为组列表
14 POST /admin/app/behavior-groups/sync - 同步行为组
```

六、数据库

6.1 表前缀规范

前缀	模块	说明
sys_	IAM	系统核心表
i18n_	I18N	国际化表
app_	APP	应用管理表

6.2 数据库脚本位置

docs/sql/ 目录下包含初始化脚本:

- 10.iam-sysyuser.sql - IAM 模块
- 11.iam-dict.sql - 字典数据
- 20.i18n.sql - 国际化模块

6.3 数据库配置

开发环境 (application-dev.yml):

代码块

```
1  spring:
2    datasource:
3      url: jdbc:mysql://120.79.30.8:3306/looi-admin-test
```

```
4     username: root
5     password: kcwl123!
```

七、部署指南

详见 40.LOOI-Deployment-Guide.md

7.1 环境信息

项目	值
服务器 IP	120.79.30.8
后端端口	8092
前端端口	8090 (Nginx)

7.2 打包命令

代码块

```
1  # 后端打包
2  cd D:\Project\looi-admin\looi-admin-server
3  mvn clean package -DskipTests
4
5  # 前端打包
6  cd D:\Project\looi-admin\looi-admin-web
7  npm run build
```

7.3 Nginx 配置

前端静态文件部署在 /usr/local/nginx/html/looi-admin-web , API 代理到 127.0.0.1:8092

八、开发规范

8.1 编码规范

遵循 .claude/rules/ 下的规范:

文件	说明
coding-style.md	代码组织、命名、方法长度
frontend-style.md	前端 UI 设计规范
security.md	安全规则 (禁止硬编码、输入校验)
testing.md	测试规范 (TDD、覆盖率门槛)

8.2 方法长度限制

层级	最大行数
Controller 方法	30 行
Service 方法	50 行
工具方法	20 行

8.3 禁止事项

- 禁止硬编码密钥、密码
- 禁止无边界校验的数值操作
- 禁止 catch (Exception e) 而不处理

九、已知问题

9.1 Token 认证问题

描述: /admin/user/info 接口硬编码返回 `userId=1` , 前端使用 mock token

状态: 未完全解决, 生产环境需实现真实 JWT 认证

9.2 测试问题

描述: 单元测试因 MyBatis Plus `baseMapper` 注入问题失败

临时解决方案: 打包时使用 `-DskipTests` 跳过测试

十、后续任务

10.1 待完成功能

- 真实 JWT 认证实现
- 用户信息从 SecurityContext 获取
- 单元测试完善

10.2 技术债务

- 老项目代码重构 (SourceCode/ 目录)
- 时间轴性能优化
- E2E 测试覆盖

10.3 优化建议

1. 考虑引入 JWT Spring Security 框架
2. 时间轴组件抽取为独立包
3. 增加行为组批量同步功能

十一、相关文档

在代码里的doc文件夹里



文档	说明
CLAUDE.md	项目架构与设计决策
40.LOOI-Deployment-Guide.md	部署指南
30.Error-Log-And-Lessons.md	踩坑记录
31.Behavior-Audit-Design.md	行为审核设计
33.Behavior-Timeline-Module-Notes.md	时间轴模块笔记

报表

报表有两个一个是dataease,一个是sqlbot，全部都放在我们的开发服务器上，

Address

47.112.161.16

General

可触未来_开发环境

Parent Group

开发

Backspace

Default

SSH on 22 port

Credentials

root

ali-dev-47.112.161.16.pem

+ SSH.id, Password, Certificate, FIDO2

Show more

Add Telnet

Connect

文档我已经交给了曾立强

https://f5d23thc7h.feishu.cn/wiki/OVjiw5zoMiee5KkJlfDcZfiwnqd

30/35



DataEase部署文档

47.112.161.16:8100/#/data/dataset

(base) [root@iZwz97ok697q29eyi1dc6rZ dataease-offline-installer-v2.10.21-ce]# /bin/bash install.sh

当前时间 : Mon Apr 20 15:17:47 CST 2026

1. 检查安装环境并初始化环境变量
全新安装
2. 设置运行目录
运行目录 /opt/dataease2.0
配置文件目录 /opt/dataease2.0/conf
3. 初始化运行目录
复制安装文件到运行目录
调整配置文件参数
复制地图文件
4. 安装 dectl 命令行工具
安装至 /usr/local/bin/dectl & /usr/bin/dectl
5. 修改操作系统相关设置
防火墙未开启, 忽略端口开放
6. 安装 docker
检测到 Docker 已安装, 跳过安装步骤
启动 Docker
7. 安装 docker-compose
检测到 Docker Compose 已安装, 跳过安装步骤
8. 加载 DataEase 镜像
加载镜像 dataease:v2.10.21
加载镜像 mysql:8.4.5
9. 配置 DataEase 服务
10. 启动 DataEase 服务

===== 安装完成 =====

系统登录信息如下:

访问地址: <http://服务器IP:8100>

用户名: admin

初始密码: DataEase@123456

SQLbot部署文档

```
(base) [root@iZwz97ok697q29eyi1dc6rZ sqlbot-offline-installer-v1.7.0-ce]# bash install.sh
```

当前时间 : Thu Mar 19 14:47:36 CST 2026

1. 检查安装环境并初始化环境变量
全新安装
2. 设置运行目录
运行目录 /opt/sqlbot
配置文件目录 /opt/sqlbot/conf
3. 初始化运行目录
复制安装文件到运行目录
调整配置文件参数
4. 安装 sctl 命令行工具
安装至 /usr/local/bin/sctl & /usr/bin/sctl
5. 修改操作系统相关设置
防火墙未开启, 忽略端口开放
6. 安装 docker
检测到 Docker 已安装, 跳过安装步骤
启动 Docker
7. 安装 docker-compose
检测到 Docker Compose 已安装, 跳过安装步骤
8. 加载 SQLBOT 镜像
加载镜像 sqlbot:v1.7.0
9. 启动 SQLBOT 服务

Network sqlbot_sqlbot-network Creating

Network sqlbot_sqlbot-network Created

Container sqlbot Creating

Container sqlbot Created

Container sqlbot Starting

Container sqlbot Started

SQLBot 服务状态 : 正常运行

===== 安装完成 =====

系统登录信息如下:

访问地址: http://服务器IP:8000

用户名: admin

初始密码: Looi@123456

DataEase文档

[DataEase - 人人可用的开源 BI 工具 - 官网](#)

SQLBot文档

[SQLBot - 基于大模型和 RAG 的智能问数系统 - 官网](#)

需求

主要是关于一些埋点的大屏幕展示, 每一个模块我都是自己书写的SQL语句, 没有使用dataease的自己的构建工具

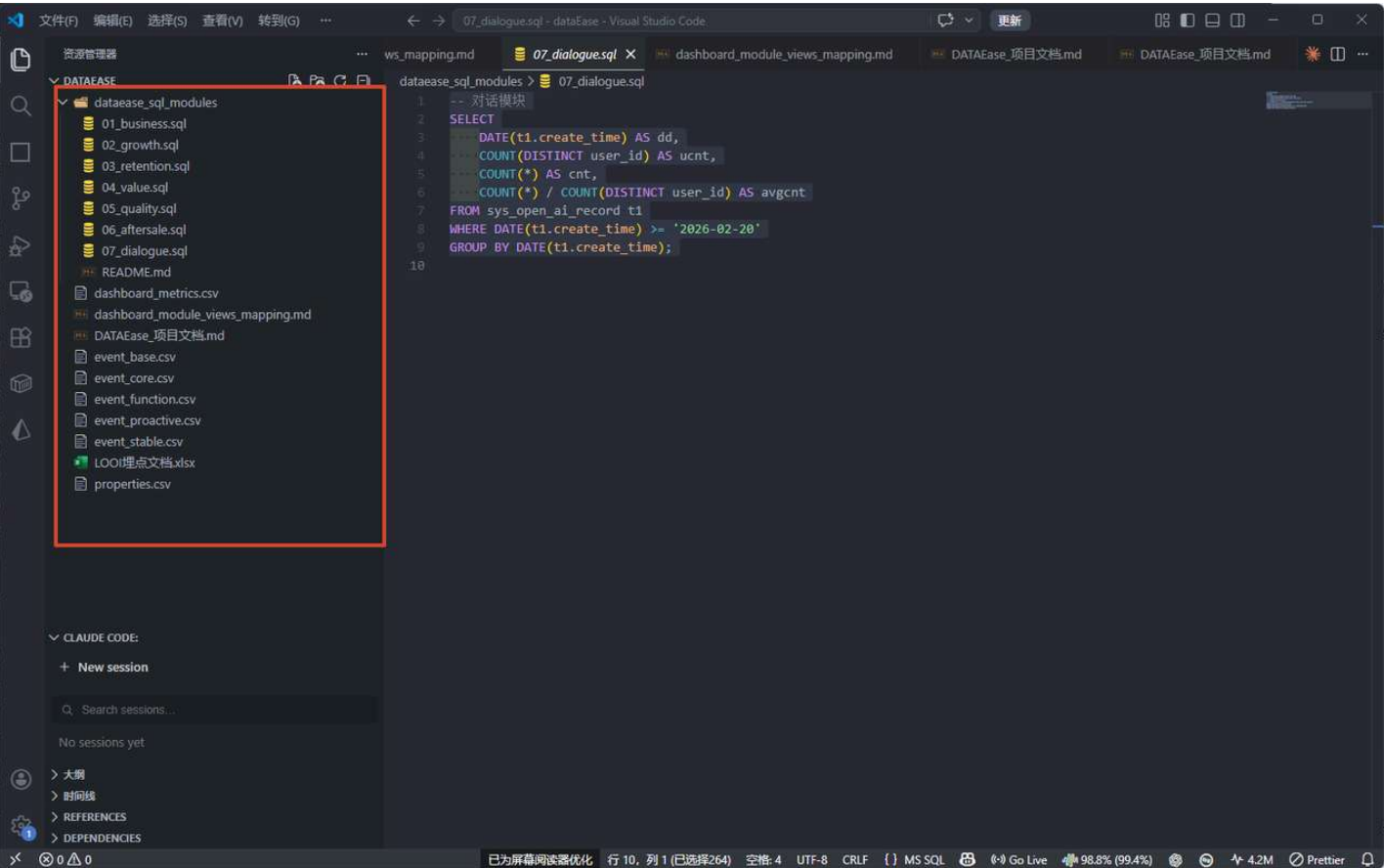
他的埋点数据在json里面, dataease没有解析json的东西, 只能自己手动写sql语句, 不太好维护

我没能实现动态数据库, 也就是国内数据库, 国外数据库两套数据库, 能够在一个报表里面动态切换,

所以我是手动建立了两个一样的报表除了数据源不同，你可以调研一番



哭了一摸一样的写两遍，一个改动也要改两个
然后是需求文档都整理在这里



简单的操作

操作就从数据源开始，也就是选择数据库，主要的数据库，国内外，正式测试4个
国内测试：

url: jdbc:mysql://120.79.30.8:3306/looi-admin-test?useUnicode=true&characterEncoding=utf-8&serverTimezone=Asia/Shanghai&useSSL=false&allowMultiQueries=true
username: root
password: kcw123!

国内正式:

```
url: jdbc:mysql://mr-sk2e8ppf6zr5qy997d.rwlb.rds.aliyuncs.com/looi-pro?useUnicode=true&characterEncoding=utf-8&serverTimezone=Asia/Shanghai&useSSL=false&allowMultiQueries=true
username: looiroot
password: Tgb#)2025!^
```

海外测试:

```
url: jdbc:mysql://looi.czwe66gc43sx.us-east-1.rds.amazonaws.com:3306/looi-test?useUnicode=true&characterEncoding=utf-8&serverTimezone=Asia/Shanghai&useSSL=false&allowMultiQueries=true
username: root
password: VGFuZ2libGVmdXR1cmUvLg==
```

海外正式:

```
url: jdbc:mysql://looi.czwe66gc43sx.us-east-1.rds.amazonaws.com:3306/looi?useUnicode=true&characterEncoding=utf-8&serverTimezone=Asia/Shanghai&useSSL=false&allowMultiQueries=true
username: root
password: VGFuZ2libGVmdXR1cmUvLg==
```

ssh配置

主机: 23.20.165.138

端口: 22

用户名: centos

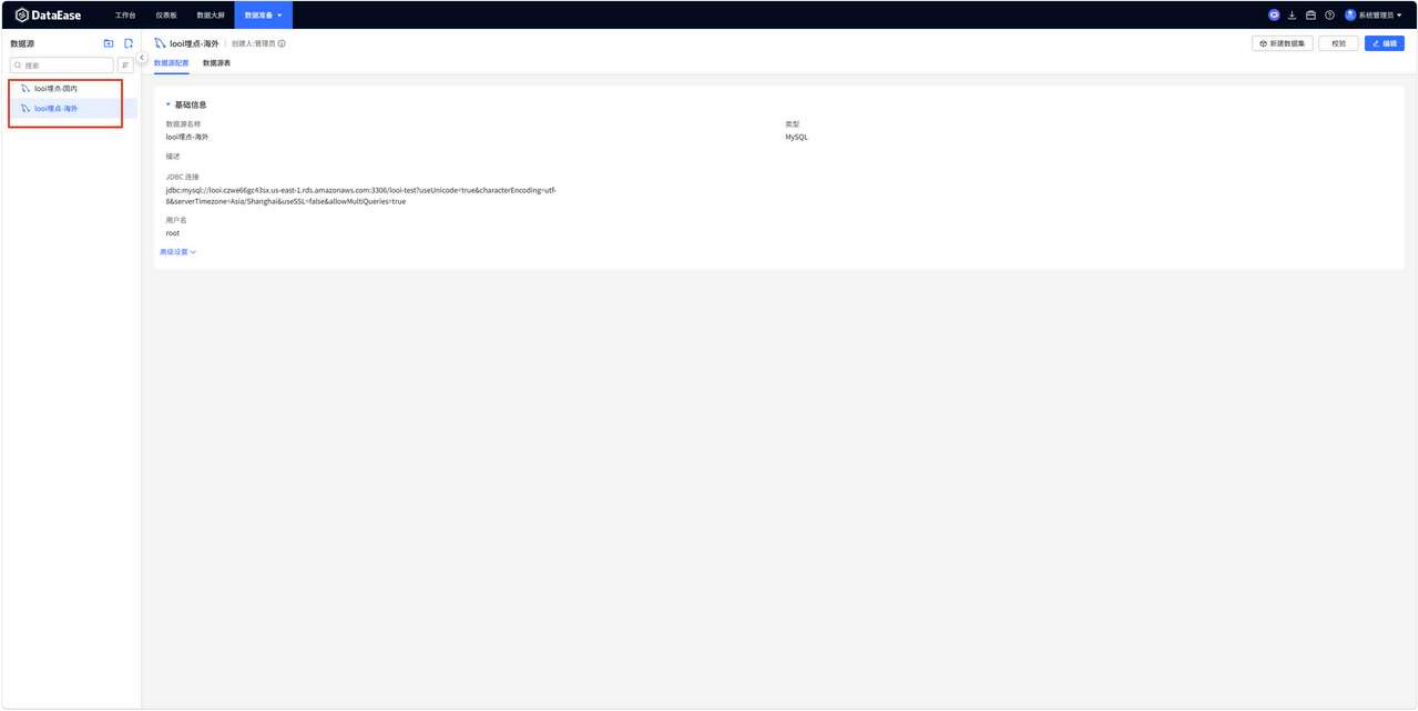
sshkey:

 aws-pro-23.20.165.138.pem
1.64KB



埋点数据表:

- event_base
- event_core
- event_function
- event_proactive
- event_stable



然后再是数据集，这是作为我们可视化数据的下一层，一个模块就是一个图块也方便了管理

